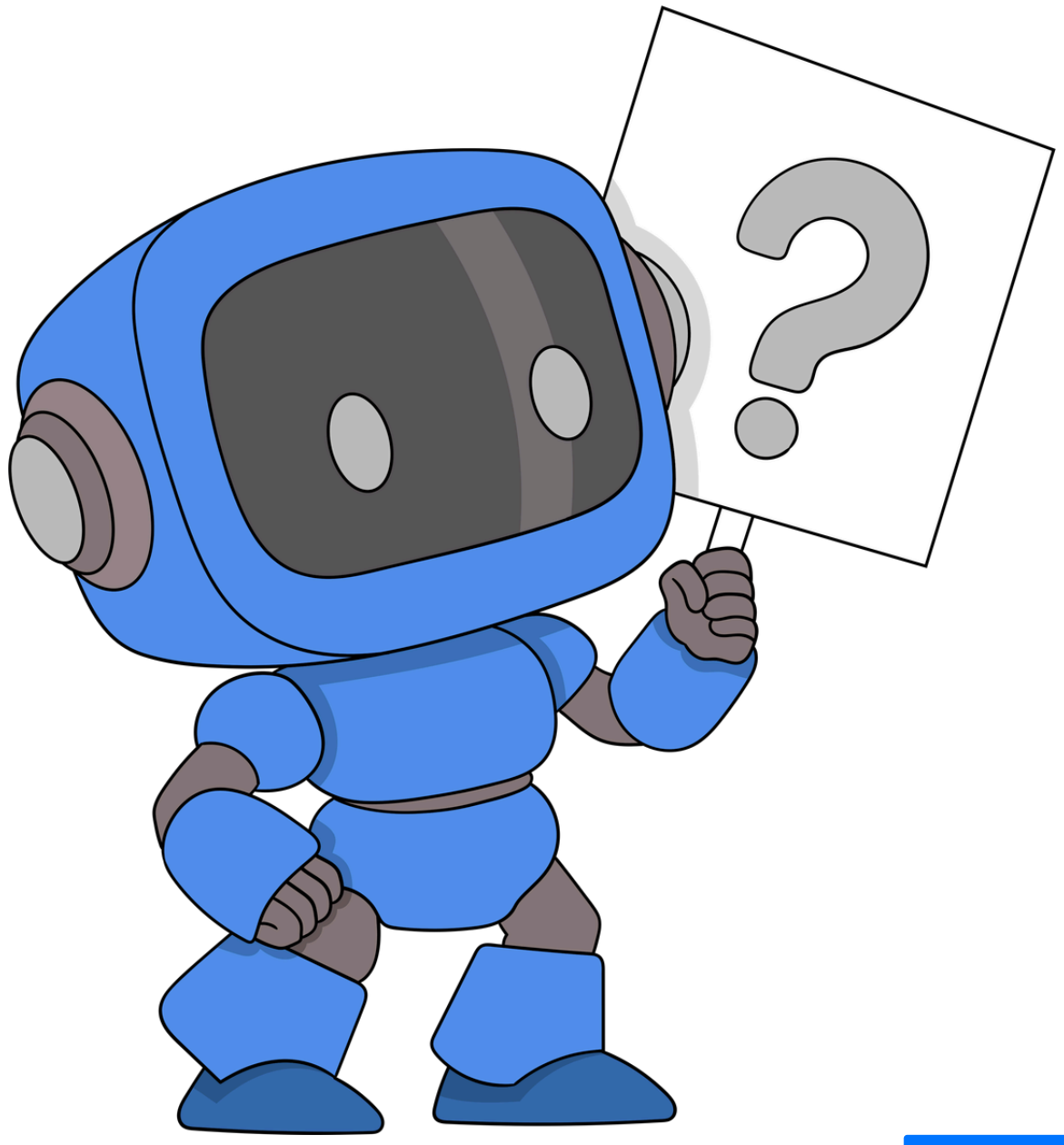
UNDERSTANDING APPROXIMATE NEAREST NEIGHBOR

The Algorithm Behind Fast Vector Search in RAG



HRUTHIK GALI DILEEP

FOLLOW

Repost

What is ANN Algorithm?

Approximate Nearest Neighbor (ANN)
Definition: ANN is an algorithm that finds data points in a dataset that are closest to a given query point, trading perfect accuracy for speed.

The Problem ANN Solves
Imagine you have a million chunks converted to vectors (embeddings). When a user asks a question, you need to find the most relevant chunks quickly. Checking all 10 million vectors would take too long.

Without ANN: Check every single vector (exact search)

- Time: Several seconds to minutes
- Accuracy: 100%
- Practical: NO

With ANN: Use smart shortcuts to find closest matches

- Time: Milliseconds
- Accuracy: 95-99%
- Practical: YES

Four Main ANN Techniques

1. HNSW (Hierarchical Navigable Small World)
2. IVF (Inverted File Index)
3. PQ (Product Quantization)
4. LSH (Locality-Sensitive Hashing)

Key Concept
ANN accepts a small accuracy trade-off (1-5% less perfect) to gain massive speed improvements (10-1000x faster).

How ANN is Used in Vector Databases

The RAG Pipeline Flow

1. Documents → Split into chunks
2. Chunks → Convert to embeddings (vectors)
3. Vectors → Store in Vector Database with ANN index
4. User Query → Convert to vector
5. ANN Search → Find similar vectors quickly
6. Retrieve → Get matching document chunks
7. LLM → Generate answer using context

Why Vector Databases Need ANN

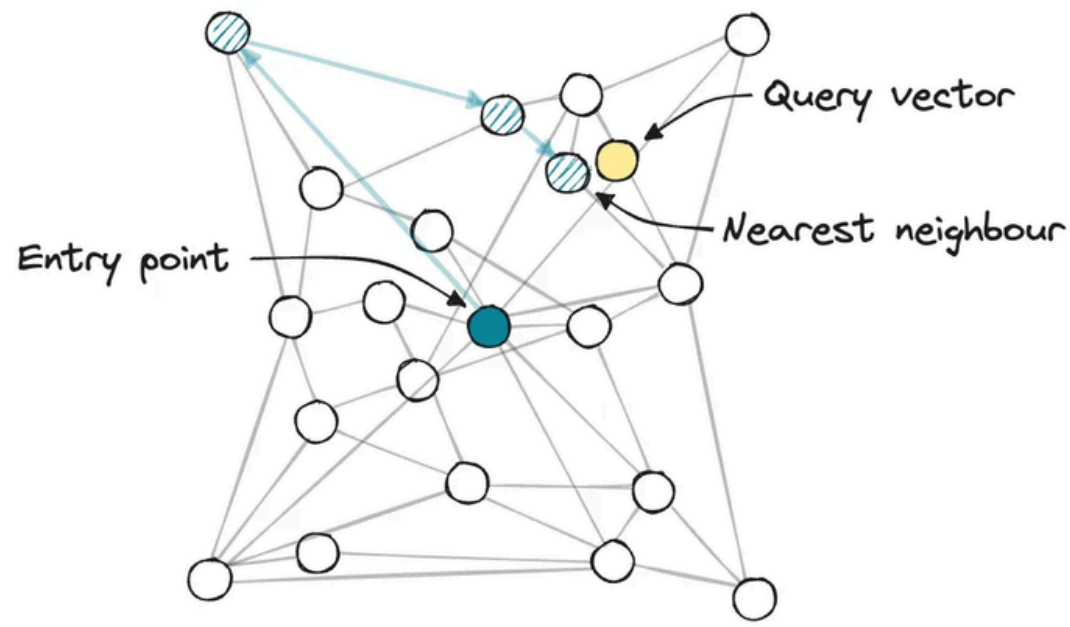
Vector databases store millions of high-dimensional vectors (typically 768 to 1536 dimensions).

Without ANN:

- Linear Search: O(n) complexity - check every vector
- Time grows linearly with database size
- Unusable at scale

With ANN:

- Sublinear Search: O(log n) or better
- Time grows slowly as database scales
- Production-ready performance



How ANN Works

The Fundamental Principle

Instead of checking every vector, ANN organizes vectors during indexing so it can skip most of them during search.

Three-Phase Process

Phase 1: Indexing (Build Time)

- Organize vectors into efficient data structures
- Create shortcuts, clusters, or graphs
- One-time cost when adding vectors
- Takes more time but enables fast searches

Phase 2: Search (Query Time)

- Navigate the index structure intelligently
- Check only promising candidate vectors
- Skip large portions of the database
- Returns approximate results quickly

Phase 3: Refinement

- Re-rank top candidates if needed
- Calculate exact distances for final results
- Return most similar vectors with confidence scores

Without ANN: Like checking every house in a city to find your friend

With ANN: Like knowing the neighborhood first, then the street, then checking a few houses

Why ANN Matters for Retrieval

Critical Benefits

1. Speed at Scale

- Handle billions of vectors in real-time
- Sub-100ms query latency even with massive datasets
- Enables production RAG systems

2. Cost Efficiency

- Less computational resources needed
- Lower cloud infrastructure costs
- Can run on standard hardware

3. User Experience

- Instant search results
- Real-time chatbot responses
- Seamless application performance

4. Scalability

- Grow from thousands to billions of documents
- Performance degrades slowly as data grows
- Horizontal scaling becomes practical

Without ANN: RAG Wouldn't Work

Modern RAG applications require:

- Instant retrieval: <100ms response times
- Large knowledge bases: Millions of documents
- High accuracy: Top-5 results must be relevant

ANN makes all three possible simultaneously. Without it, you'd have to choose between speed and scale.

Vector Databases Offering ANN

Pinecone

- Fully managed cloud service
- HNSW-based algorithm
- Auto-scaling and serverless options
- Best for: Production systems, minimal maintenance

Weaviate

- Open-source with managed option
- HNSW algorithm
- Built-in ML models and hybrid search
- Best for: Complex queries, multi-modal search

Qdrant

- Rust-based for high performance
- HNSW with advanced filtering
- Distributed architecture
- Best for: High-throughput applications

Milvus

- Open-source, CNCF project
- Multiple ANN algorithms (IVF, HNSW, DiskANN)
- GPU acceleration support
- Best for: Large-scale enterprise deployments

Chroma

- Developer-friendly, embedded mode
- HNSW algorithm
- Simple API, quick setup
- Best for: Development and prototyping

FAISS (by Meta)

- Research library, not a full database
- Multiple algorithms (IVF, HNSW, PQ)
- Highly optimized
- Best for: Custom implementations, research

Hierarchical Navigable Small World Graphs

What It Is

HNSW builds a multi-layer graph structure where:

- Each vector is a node in the graph
- Nodes connect to their nearest neighbors
- Multiple layers exist, from coarse to fine
- Search starts at the top and drills down

How It Works

Building the Index:

1. Create multiple layers (like a pyramid)
2. Top layers: Few nodes, long-range connections
3. Bottom layers: All nodes, short-range connections
4. Connect each node to M nearest neighbors per layer

Searching:

1. Start at the top layer (sparse, quick navigation)
2. Find nearest node in that layer
3. Move down to the next layer
4. Repeat until reaching the bottom
5. Perform local search in the densest layer

When to Use

Use HNSW when:

- You need excellent recall (95-99% accuracy)
- Query speed is critical (<10ms)
- Memory is not a constraint
- You have frequent updates to the database

Don't use when:

- Memory is severely limited
- Dataset has billions of vectors (consider IVF)
- Insertion speed matters more than query speed



Inverted File Index

What It Is

IVF partitions the vector space into clusters (like organizing books by genre) and searches only relevant clusters instead of the entire database.

How It Works

Building the Index:

1. Use k-means clustering on all vectors
2. Create N clusters (typically 100-10,000)
3. Assign each vector to its nearest cluster center
4. Build an inverted index mapping clusters to vectors

Searching:

1. Find the query vector's nearest cluster centers
2. Search only within those few clusters (nprobe parameter)
3. Check vectors only in selected clusters
4. Return top-k most similar vectors

Key Parameter - nprobe:

- nprobe=1: Search only 1 cluster (fastest, least accurate)
- nprobe=10: Search 10 clusters (balanced)
- nprobe=100: Search 100 clusters (slower, more accurate)

When to Use

Use IVF when:

- Dataset has billions of vectors
- Memory is limited (can be disk-backed)
- You can tolerate slightly lower recall (90-95%)
- Batch queries are common

Don't use when:

- Dataset is small (<100K vectors)
- You need highest possible recall
- Real-time updates are critical
- Cluster distribution is highly skewed

Product Quantization (PQ)

What It Is

PQ compresses vectors into smaller representations using quantization, reducing memory usage by 10-30x while maintaining reasonable accuracy.

How It Works

Compression Process:

1. Split each vector into M sub-vectors (e.g., 1536-dim → 8 sub-vectors of 192-dim)
2. For each sub-space, create a codebook of 256 centroids
3. Replace each sub-vector with the ID of its nearest centroid
4. Store only the IDs (8 bytes instead of 6144 bytes)

Searching:

1. Pre-compute distances from query sub-vectors to all codebook centroids
2. For each database vector, look up its centroid IDs
3. Sum the pre-computed distances
4. Return vectors with smallest total distances

Math: Original 1536 floats (6KB) → 8 bytes (compression ratio: 768:1)

When to Use

Use PQ when:

- Memory is the bottleneck
- You have millions/billions of vectors
- Moderate accuracy is acceptable (85-90%)
- Combined with IVF (IVF-PQ is powerful)

Don't use when:

- Memory is plentiful
- You need maximum accuracy
- Vector dimensionality is very low (<100)
- Fast insertion is critical

Locality-Sensitive Hashing

What It Is

LSH uses special hash functions that map similar vectors to the same hash bucket with high probability, turning similarity search into a hash lookup problem.

How It Works

Building the Index:

1. Create multiple hash functions (hash tables)
2. Each hash function groups similar vectors together
3. Store vectors in buckets based on their hash values
4. Use multiple hash tables to improve recall

Searching:

1. Hash the query vector with the same functions
2. Look up the buckets in all hash tables
3. Collect vectors from matching buckets
4. Rank candidates by actual distance
5. Return top-k results

Key Insight: Hash collisions are GOOD - they indicate similarity!

When to Use

Use LSH when:

- You have extremely high-dimensional data (>1000 dims)
- You need simple, easy-to-implement solution
- You're working with sparse vectors
- Memory efficiency is important

Don't use when:

- You need the highest recall possible
- Dataset is small (<10K vectors)
- You have access to better algorithms (HNSW, IVF)
- Query speed is more critical than simplicity